Contextual Embeddings: ELMo, USE, BERT

Lecture 9: Beyond Static Word Representations

LLM Course - Week 4

Winter 2026

Today's Lecture 🤔

1. 🤔 From Static to Contextual
2. 🤔 Language Models as Feature Extractors
3. 🤔 ELMo: Embeddings from Language Models
4. 🌐 Universal Sentence Encoder
5. 🤔 BERT: Bidirectional Transformers
6. 🤔 Comparison & Applications

Goal: Understand how context transforms word representation

The Polysemy Problem Revisited 🤔

Recall: Static embeddings assign ONE vector per word

Example: ”bank”

1. ”I deposited money at the **bank**” (financial institution)
2. ”We sat by the river **bank**” (riverside)
3. ”The plane will **bank** left” (tilt/turn)

Word2Vec/GloVe: All three get the same vector 🤔

Contextual embeddings: Each gets a different vector based on context 🤔

Static vs. Contextual Embeddings 🤔

Static (Word2Vec, GloVe, FastText):

```
# Same vector every time
model = Word2Vec(...)
vec1 = model['bank']
vec2 = model['bank']

assert vec1 == vec2  # True!
```

Characteristics:

- One vector per word type
- Context-independent
- Fast lookup (dictionary)
- Fixed after training

Contextual (ELMo, BERT):

```
# Different vector per
    occurrence
model = BertModel.
    from_pretrained('bert-base')

sent1 = "river bank"
sent2 = "money bank"

vec1 = get_embedding(model,
    sent1, 'bank')
vec2 = get_embedding(model,
    sent2, 'bank')

assert vec1 != vec2  # True!
```

Language Models as Feature Extractors

Key Insight: Train a language model, use its internal states as embeddings

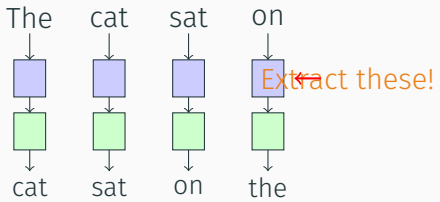
Language Modeling Task:

Predict the next word given previous words

$$P(w_t|w_1, w_2, ..., w_{t-1})$$

Why This Works:

- To predict next word, model must understand:
  - Syntax (grammar)
  - Semantics (meaning)
  - Context (discourse)
- Hidden states encode this

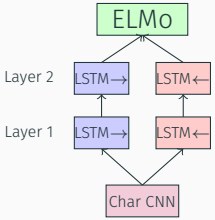


ELMo: Embeddings from Language Models

The first widely-adopted contextual embedding (2018)

Key Ideas:

1. Train deep bidirectional LSTM language model
2. Use all layer activations
3. Weighted combination per task
4. Pre-train on large corpus



Architecture:

- 2-layer biLSTM
- Forward LM: $P(w_t|w_1...w_{t-1})$
- Backward LM: $P(w_t|w_{t+1}...w_n)$

ELMo representation:

$$\text{ELMo}_k^{task} = \gamma^{task} \sum_{j=0}^{L} s_j^{task} \mathbf{h}_{k,j}$$

ELMo: How It Works

Training:

1. Pre-train on large corpus (1B Word Benchmark)
2. Minimize combined forward & backward LM loss:

$$\mathcal{L} = \sum_{k=1}^{N} (\log P(w_k | w_1 ... w_{k-1}) + \log P(w_k | w_{k+1} ... w_N))$$

3. Each position gets representation from all layers

Usage (downstream tasks):

1. Freeze ELMo weights
2. For each token, compute:
   - Character-based representation
   - 2 forward LSTM hidden states
   - 2 backward LSTM hidden states

ELMo in Practice

```
from allennlp.modules.elmo import Elmo, batch_to_ids

# Initialize ELMo
options_file = "https://s3-us-west-2.amazonaws.com/allennlp/models
    /elmo/2x4096_512_2048cnn_2xhighway/
    elmo_2x4096_512_2048cnn_2xhighway_options.json"
weight_file = "https://s3-us-west-2.amazonaws.com/allennlp/models/
    elmo/2x4096_512_2048cnn_2xhighway/
    elmo_2x4096_512_2048cnn_2xhighway_weights.hdf5"

elmo = Elmo(options_file, weight_file, 2, dropout=0)

# Prepare sentences
sentences = [
    ['I', 'deposited', 'money', 'at', 'the', 'bank'],
    ['We', 'sat', 'by', 'the', 'river', 'bank']
```

Universal Sentence Encoder (USE) 🌐

Sentence-level embeddings for semantic similarity

Motivation:

- Word embeddings: good for words
- But what about sentences?
- Average of word vectors? Too simple!
- Need compositionality

Two Variants:

1. Transformer-based:

- Higher accuracy
- Slower (compute intensive)

Training Objectives:

1. Unsupervised: Skip-thought
2. Supervised: SNLI (entailment)
3. Multi-task learning

Output:

- 512-dimensional vector
- Fixed-length (any sentence length)
- Optimized for similarity tasks

Use Cases:

- Semantic search

Universal Sentence Encoder in Practice

```
import tensorflow_hub as hub
import numpy as np

# Load model
embed = hub.load("https://tfhub.dev/google/universal-sentence-
    encoder/4")

# Example sentences
sentences = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "The dog ran in the park.",
    "I love machine learning."
]

# Generate embeddings
```

The model that changed everything (2018)

Key Innovations:

1. Bidirectional context (not just left-to-right)
2. Transformer architecture (attention)
3. Masked language model pre-training
4. Next sentence prediction
5. Deeply bidirectional

Impact:

Architecture Sizes:

Model	Layers	Hidden
BERT-Base	12	768
BERT-Large	24	1024

Training Data:

- BooksCorpus (800M words)
- English Wikipedia (2.5B words)
- Total: 3.3B words

Masked Language Modeling (MLM)

BERT's key training innovation

The Problem with Traditional LM:

- Left-to-right: Only sees previous words
- Right-to-left: Only sees next words
- Want: See both directions simultaneously
- But: Can't just show the answer during training!

Solution: Mask some words, predict them

Example
Original: "The cat sat on the mat"

Masked: "The [MASK] sat on the mat"

Task: Predict [MASK] = "cat"

Next Sentence Prediction (NSP) [?]

Second pre-training task: Understand sentence relationships

Task: Given two sentences A and B, predict if B follows A in the text

Positive Example (IsNext)
Sentence A: "The cat sat on the mat."

Sentence B: "It was sleeping peacefully."

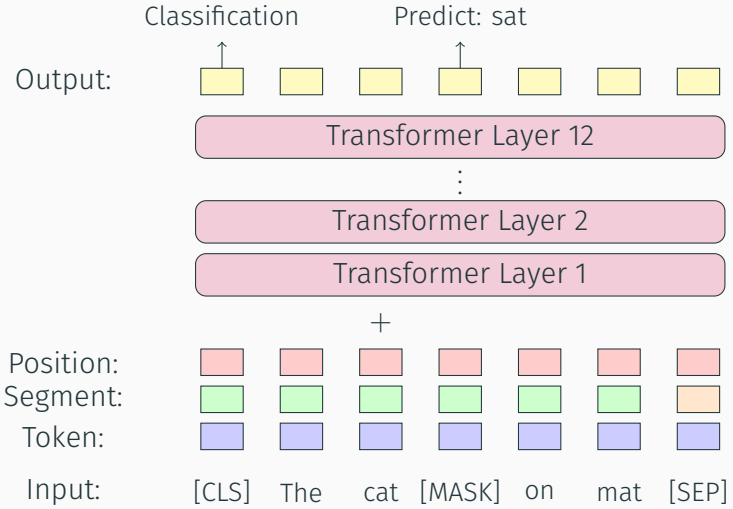
Label: IsNext [?]

Negative Example (NotNext)
Sentence A: "The cat sat on the mat."

Sentence B: "Machine learning is fascinating."

Label: NotNext [?]

BERT Architecture



BERT in Practice

```python
from transformers import BertTokenizer, BertModel
import torch

# Load pre-trained BERT
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

# Example sentences with "bank"
sent1 = "I deposited money at the bank"
sent2 = "We sat by the river bank"

# Tokenize
tokens1 = tokenizer(sent1, return_tensors='pt')
tokens2 = tokenizer(sent2, return_tensors='pt')

# Get embeddings
```

Fine-tuning BERT 🤗

Two ways to use BERT:

1. Feature Extraction:
   - Freeze BERT weights
   - Use embeddings as features
   - Train classifier on top
   - Faster, less data needed

```
# Freeze BERT
for param in bert_model.
    parameters():
    param.requires_grad = False

# Add classifier
```

2. Fine-tuning:
   - Update BERT weights
   - Add task-specific head
   - Train end-to-end
   - Better performance, more data needed

```
# Keep BERT trainable
bert_model = BertModel.
    from_pretrained(
    'bert-base-uncased'
)
```

Contextual Embeddings Comparison [?]

Property	ELMo	USE	BERT
Architecture	BiLSTM	Transformer/DAN	Transformer
Pre-training	LM (forward+backward)	Multi-task	MLM + NSP
Bidirectional	Shallow	Yes	Deep
Granularity	Token	Sentence	Token
Hidden size	1024	512	768/1024
Parameters	93M	256M	110M/340M
Speed	Medium	Fast	Slow
OOV handling	Characters	Subwords	WordPiece
Year	2018	2018	2018

Recommendations

- **ELMo:** Legacy systems, character-aware needs

Impact on NLP

BERT revolutionized NLP:

Before BERT (pre-2018):

- Task-specific architectures
- Train from scratch
- Static embeddings (Word2Vec, GloVe)
- Limited transfer learning
- Moderate performance

Tasks that improved:

- Question Answering (+1.5 F1 on SQuAD)

After BERT (post-2018):

- Pre-train, then fine-tune
- Transfer learning standard
- Contextual embeddings
- Massive pre-trained models
- State-of-the-art results

BERT Variants:

- RoBERTa: Better training
- ALBERT: Parameter sharing
- DistilBERT: Smaller, faster

Real-World Applications

1. **Search & Information Retrieval:**
   - Google Search uses BERT for query understanding
   - Semantic search engines
   - Document ranking
2. **Question Answering:**
   - Reading comprehension (SQuAD)
   - Customer support chatbots
   - FAQ systems
3. **Text Classification:**
   - Sentiment analysis
   - Intent detection
   - Content moderation
4. **Named Entity Recognition:**
   - Extract people, places, organizations
   - Medical entity extraction
   - Legal document processing

Do contextual embeddings truly ”understand” language?

Consider:

- BERT can distinguish ”bank” (financial) from ”bank” (riverside)
- It achieves human-level performance on many benchmarks
- But it’s trained only on text co-occurrence patterns

Arguments For:

- Captures complex semantic relationships
- Generalizes to new contexts
- Emergent linguistic capabilities

Arguments Against:

- No grounding in physical world
- No common sense reasoning
- Exploits statistical shortcuts
- Brittle to adversarial examples

Practical Tips

1. **Choosing a Model:**
   - BERT-base: Good balance, 110M params
   - DistilBERT: 40% smaller, 60% faster, 97% performance
   - RoBERTa: Better than BERT, longer training
   - Domain-specific: BioBERT, SciBERT, FinBERT, etc.
2. **Fine-tuning Best Practices:**
   - Small learning rate (2e-5 typical)
   - Few epochs (2-4)
   - Batch size: 16 or 32
   - Warm-up steps
   - Gradient clipping
3. **Computational Considerations:**
   - BERT-base: 110M params, 512 max tokens
   - Needs GPU (1-4 GB VRAM minimum)
   - Batching for efficiency
   - Consider DistilBERT for production

Summary

What we learned today:

1. **Contextual vs. Static:** Different vectors per occurrence
2. **ELMo (2018):**
   - BiLSTM language models
   - Character-based, handles OOV
   - Task-specific weighting
3. **Universal Sentence Encoder (2018):**
   - Sentence-level embeddings
   - Two variants: Transformer & DAN
   - Optimized for semantic similarity
4. **BERT (2018):**
   - Masked language modeling
   - Deep bidirectional transformers
   - Pre-train + fine-tune paradigm
   - Revolutionized NLP

Key References

Foundational Papers:

- Peters et al. (2018). ”Deep contextualized word representations” (ELMo)
- Cer et al. (2018). ”Universal Sentence Encoder”
- Devlin et al. (2018). ”BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”

BERT Variants:

- Liu et al. (2019). ”RoBERTa: A Robustly Optimized BERT Pretraining Approach”
- Sanh et al. (2019). ”DistilBERT, a distilled version of BERT”
- Lan et al. (2019). ”ALBERT: A Lite BERT for Self-supervised Learning”

Critical Perspectives:

- Bender & Koller (2020). ”Climbing towards NLU: On Meaning, Form, and

Next Lecture:

Dimensionality Reduction: PCA, t-SNE, UMAP

Visualizing high-dimensional embeddings!